Commutator-Graph Term Ordering Reduces First-Order Trotter Error at a Fixed Gate Budget

Molena Huynh
North Carolina State University
molena.huynh@jmp.com

June 24, 2026

Abstract

The order in which terms are applied in a first-order (Lie–Trotter) product formula is a free parameter: the per-step gate count of a Hamiltonian $H = \sum_j c_j P_j$ is one rotation per Pauli term regardless of order, yet the algorithmic error depends on that order through the commutators of the non-commuting summands. Term ordering can therefore lower Trotter error at *zero* additional gate cost. We treat the terms of H as the nodes of a *commutator graph*—an edge connects two terms that anticommute—and order them with a greedy policy that minimizes the coefficient-weighted number of anticommuting adjacencies in the schedule; we also train a small interpretable router (logistic regression over eight commutator-graph features) that selects among fixed heuristics per instance. Across 120 structured Hamiltonian instances (transverse-field Ising, Heisenberg, and random molecular-like families, 4–8 qubits) evaluated against exact statevector references, commutator-aware ordering raises mean Trotter fidelity from 0.205 ± 0.040 (random ordering) to 0.548 ± 0.068 (95% confidence intervals) at a fixed gate-budget proxy of 48—a $1.76\times$ reduction in mean infidelity—while cutting the mean number of adjacent anticommuting pairs from 3.08 to 0.28. The clear, statistically robust effect is ordering versus random; among the non-random heuristics the fidelity differences (commutator 0.548, coefficient 0.550, learned router 0.549) lie within the confidence intervals and are *not* distinguishable at this scale, and the router tracks rather than beats the best fixed strategy. This is a controlled, fully reproducible demonstration that term ordering is a real, no-cost lever on Trotter error—deliberately small, first-order, and classically simulated—not a claim of hardware advantage.

1 Introduction

Simulating the time evolution $e^{-\mathrm{i}Ht}$ of a quantum many-body Hamiltonian is one of the canonical applications of quantum computers and the original motivation for the field [1, 2]. The most widely deployed simulation primitive is the product formula. For a Hamiltonian written in its Pauli decomposition,

$$H = \sum_{j=1}^{L} c_j P_j, \tag{1}$$

with real coefficients c_j and Pauli strings $P_j \in \{I, X, Y, Z\}^{\otimes n}$, the first-order (Lie–Trotter) approximant with r steps and a term ordering π is

$$U_{\mathrm{Trot}}(\pi) = \left(\prod_{j \in \pi} e^{-\mathrm{i}\, c_j P_j\, t/r} \right)^r. \tag{2}$$

Its error against the exact propagator is controlled by the commutators of the summands: the leading correction to a single step is $\frac{t^2}{2r}\sum_{a<b}[c_aP_a, c_bP_b]$, which *vanishes for any pair of terms that commute* [3–5]. The modern, tight analysis of this error—"Theory of Trotter Error with Commutator Scaling"—makes the dependence on nested commutators explicit and is the theoretical anchor of this work [5, 6].

This paper also sits in an ordered sequence of resource-accounted graph and quantum-learning studies. The query-budget discipline follows graph-conditioned trust-region QAOA [20], while the controlled relabeling-invariant benchmark style follows the depth-one topology-conditioned QAOA study [22], and complementary learned residual Trotter corrections [21]. More directly, the Hamiltonian-simulation setting is informed by recent fixed-depth and representation-theoretic analyses by Nguyen and Jing: second-order Zassenhaus factorization isolates non-commutative corrections, and root-activity complexity bounds make Lie-algebraic structure quantitative [23, 24]. The present contribution is different and narrower: it changes only the ordering of first-order product formula terms at matched gate count.

Two facts about Eq. (2) motivate this paper. First, the *implementation cost* of a first-order schedule is one rotation $e^{-\mathrm{i}c_jP_jt/r}$ per term per step, so the gate count is $L \cdot r$ *independent of the ordering* π. Second, the *error* does depend on π, because the leading commutator sum is sensitive to which pairs of non-commuting terms are placed adjacently in the product. The ordering is therefore a free knob: it can lower the Trotter error *at no extra gate cost*. While higher-order formulas, randomized compilation, and step-size selection are the usual routes to cheaper simulation [4, 7–9], those either change the gate budget or the algorithm; reordering does not.

The structural object that exposes which pairs of terms contribute to the leading error is the *commutator graph* (also called the anticommutation graph): one node per Pauli term, with an edge between two terms whenever they anticommute. This is the same graph used in measurement-grouping for variational algorithms, where a clique cover into mutually commuting sets reduces the number of measurement settings [10–12]. Here we repurpose it for *time evolution*: a schedule that keeps anticommuting (graph-adjacent) terms apart and clusters commuting terms together minimizes the number of error-contributing junctions. The impact of Trotter ordering on chemistry simulations has been observed empirically [9, 13]; our contribution is a graph-centric formalization, a coefficient-weighted greedy ordering with an exact matched-cost evaluation, and an honest, statistically reported quantification of how large—and how robust—the effect is across structured families.

Contributions and scope. We make three contributions. (i) We formalize first-order Trotter term ordering as minimization of the coefficient-weighted count of anticommuting adjacencies on the commutator graph, and give a greedy policy that achieves a near-minimal count at no gate-cost change (Methods). (ii) We measure the effect against exact statevector references on 120 structured Hamiltonian instances and report every quantity as a mean with a 95% confidence interval, separating the statistically clear effect (ordering vs. random) from the within-noise comparison among non-random heuristics. (iii) We add a small *interpretable* learned router—logistic regression over eight commutator-graph features, trained leave-one-instance-out—and show it *tracks* the best fixed strategy rather than beating it. We state the limitations plainly: the study is first-order only, on small systems ($n \leq 8$) where an exact reference is tractable, on synthetic/structured Hamiltonians rather than production molecules, with a gate-count *proxy* ($L \cdot r$); and the differences among the non-random orderings are within the confidence intervals at this scale. The contribution is a clean, reproducible characterization of a no-cost lever, not a claim of advantage.

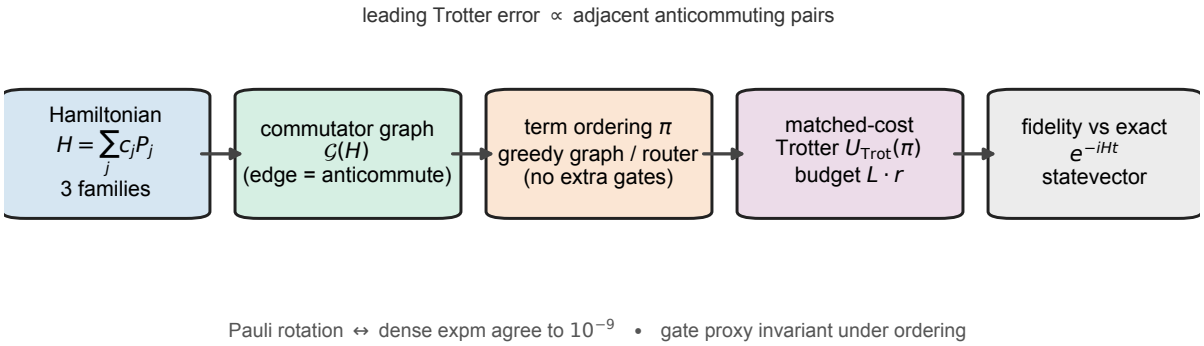

Figure 1: **Overview of the commutator-graph term-ordering benchmark.** A structured Hamiltonian instance, drawn from one of three families (transverse-field Ising, Heisenberg, random molecular-like), is decomposed into its Pauli terms $H = \sum_j c_j P_j$. The *commutator graph* $\mathcal{G}(H)$ places one node per term and an edge between every anticommuting pair (an $O(n)$ parity test). A term ordering π is then fixed either by a greedy walk on $\mathcal{G}(H)$ or by a learned router over eight graph features; crucially, reordering changes no gate, so the gate-budget proxy $L \cdot r$ is invariant. The first-order Trotter schedule $U_{\mathrm{Trot}}(\pi)$ is evaluated by an exact, matrix-free statevector simulator, and quality is scored as the fidelity against the exact propagator $e^{-\mathrm{i}Ht}$ at the identical gate budget. Two internal guarantees underpin the benchmark: the fast Pauli-rotation kernel and dense `expm` agree to 10^{-9} before any fidelity is emitted, and the leading first-order Trotter error is proportional to the number of adjacent anticommuting pairs in the schedule (Theorem 6)—the ordering-only quantity the commutator policy minimizes.

2 Results

Figure 1 summarizes the end-to-end pipeline that the following sections quantify.

2.1 Commutator-graph ordering and the matched-cost protocol

Each Hamiltonian instance is decomposed as in Eq. (1) and represented by its commutator graph $\mathcal{G}(H)$, whose nodes are the L terms and whose edges join anticommuting pairs (an $O(n)$ parity test per pair; Methods). We compare five term orderings, all evaluated at an *identical* gate budget so that any fidelity difference is attributable solely to ordering:

- **random**: a seeded shuffle (the baseline);
- **coefficient**: terms sorted by $|c_j|$ descending (a common heuristic);
- **locality**: terms sorted by their qubit support;
- **commutator** (the contribution): a greedy walk on $\mathcal{G}(H)$ that, starting from the largest-coefficient term, repeatedly appends an unused term that commutes with the current tail if possible, and otherwise the anticommuting term with the smallest coefficient product $|c_{\mathrm{tail}}|\,|c_j|$ (Methods);
- **learned**: a logistic-regression router over eight commutator-graph features that predicts which fixed strategy will give the highest fidelity and returns that strategy's ordering, trained leave-one-instance-out so no test instance informs its own predictor.

The implementation cost is the gate proxy $L \cdot r$, which is invariant under reordering; we report it alongside every comparison. The Trotter state from Eq. (2) is produced by an exact statevector simulator built from single-Pauli rotations ($e^{-\mathrm{i}\theta P} = \cos\theta\, I - \mathrm{i}\sin\theta\, P$, valid because $P^2 =$

I), validated against dense `scipy.linalg.expm` to 10^{-9} before any fidelity number is emitted (Methods). Fidelity is the squared overlap $|\langle\psi_{\text{exact}}|\psi_{\text{Trot}}\rangle|^2$ against the exact reference $e^{-\mathrm{i}Ht}|\psi_0\rangle$.

The matched-cost protocol rests on three formal guarantees proved in the Methods, which we state here so the reader can connect each experiment to the theory it illustrates. First, the gate budget $L \cdot r$ is invariant under reordering (Proposition 3), so every column of Table 1 and every point of Figure 2 compares orderings at *identical* cost. Second, the leading first-order error is, to order t^2/r, the coefficient-weighted sum over the anticommuting edges of $\mathcal{G}(H)$ (Theorem 6 and Lemma 4); this is the quantity the "adj. anticomm." column of Table 1 measures and the commutator policy minimizes. Third, the learned router's choice depends only on the intrinsic weighted graph, not on the term listing (Proposition 7), which is why its row in Table 1 tracks the best fixed strategy. The exact-simulation kernel itself is justified by Lemmas 1 and 2 and cross-checked numerically to 10^{-9} (the `pauli_algebra_verified` integrity gate of Fig. 1).

2.2 Ordering versus random: a large, statistically clear effect

Table 1 reports, for each ordering at the fixed step budget ($t = 2.0$, $r = 3$; mean gate proxy 48), the mean Trotter fidelity with its 95% confidence interval, the mean infidelity, the mean number of adjacent anticommuting pairs in the schedule, and the gate proxy. Statistics are over 120 instances (three families $\times$ five sizes $n \in \{4, \ldots, 8\}$ $\times$ eight seeded draws).

Table 1: **First-order Trotter fidelity by term ordering at a matched gate budget.** Reported scale (`configs/full.yaml`): 120 instances, mean gate proxy = 48, evolution time $t = 2.0$, $r = 3$ steps. Values are means over instances; "±95%" is the half-width of the 95% confidence interval; "adj. anticomm." is the mean number of adjacent anticommuting pairs in the schedule; "gate proxy" is $L \cdot r$, invariant under reordering. Generated by `submission/code` into `results/summary.json` and reproduced verbatim here.

ordering	fidelity	±95%	infidelity	adj. anticomm.	gate proxy
random	0.2054	0.0400	0.7946	3.08	48
coefficient	0.5501	0.0670	0.4499	2.86	48
locality	0.5004	0.0684	0.4996	6.47	48
commutator	0.5481	0.0677	0.4519	0.28	48
learned	0.5486	0.0670	0.4514	2.50	48

Two readings of Table 1 matter and must be kept separate. *First, the ordering-versus-random effect is large and statistically clear.* Random ordering achieves a mean fidelity of 0.205 ± 0.040; every non-random ordering more than doubles it, with the commutator ordering at 0.548 ± 0.068. These intervals are disjoint, and the 0.343 mean fidelity gain is many confidence-interval widths. Equivalently, the commutator ordering reduces the mean infidelity from 0.795 to 0.452, a 1.76$\times$ reduction, at the identical gate proxy of 48. The mechanism is visible in the last column: commutator ordering leaves only 0.28 adjacent anticommuting pairs on average, against 3.08 for random and 6.47 for the locality heuristic—an order-of-magnitude reduction in exactly the schedule junctions that generate the leading Trotter error.

Second, among the non-random orderings the fidelity differences are within noise. The commutator (0.548), coefficient (0.550), and learned (0.549) means lie within 0.002 of one another, far inside their $\approx \pm 0.067$ confidence intervals; they are *not* statistically distinguishable at this scale. The honest statement is therefore that commutator-aware ordering is *among the best* strategies and reaches that performance with by far the fewest anticommuting adjacencies (a directly interpretable, ordering-only surrogate for the error), but we do not claim it significantly outperforms the coefficient heuristic on fidelity here. The locality heuristic is a partial exception: it underperforms the other non-random orderings (0.500 ± 0.068) and, tellingly, has the

most anticommuting adjacencies of any policy, illustrating that ordering by qubit support can actively place non-commuting terms together.

2.3 The gate-budget frontier

Figure 2 traces the fidelity–vs–gate-budget frontier as the Trotter step count r sweeps the grid $\{1, 2, 3, 4, 6, 8, 12\}$; the x-axis is the mean gate proxy $L \cdot r$. The random ordering sits below every non-random ordering at *every* budget on the curve, and the gap is largest in the intermediate-budget regime that practitioners care about: at a mean proxy of 38 ($r = 3$) random reaches only 0.193 while the commutator and coefficient orderings reach 0.548 and 0.550; at proxy 76 ($r = 6$) the values are 0.589 (random) versus 0.823 (commutator). All orderings converge toward unit fidelity as r grows—ordering buys error reduction at fixed budget, not a change in the asymptotic rate—so the practical value is reaching a target fidelity at a *smaller* budget, or a higher fidelity at a *fixed* budget. The commutator and learned curves coincide because the router selects the commutator strategy for the frontier sweep; the coefficient curve overlaps them to within line width, again consistent with the within-noise reading.

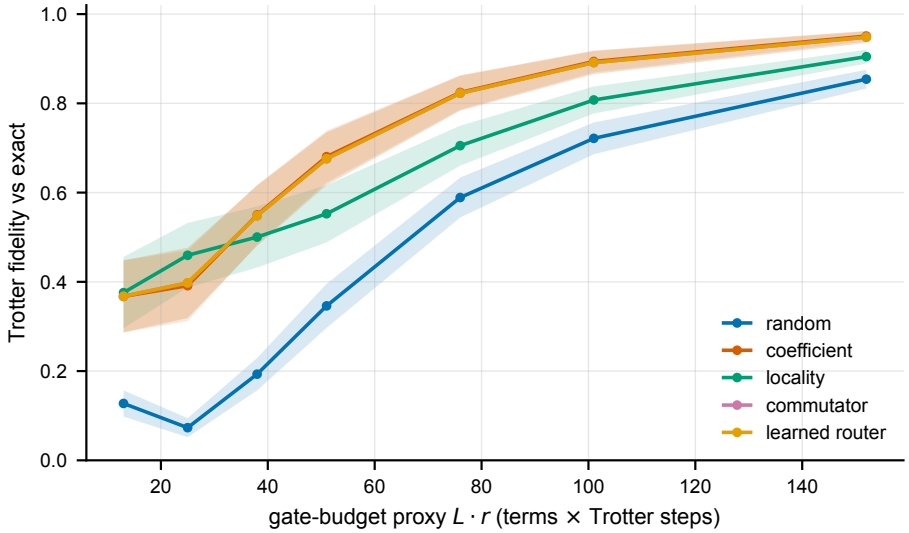

Figure 2: **Fidelity–gate-budget frontier across term orderings.** Fidelity versus gate-budget proxy ($L \cdot r$) for the five term orderings as the Trotter step count sweeps $\{1, 2, 3, 4, 6, 8, 12\}$. Lines are means over the $n = 120$ instances; shaded bands are 95% confidence intervals ($1.96\, s/\sqrt{n}$) on each mean, where s is the sample standard deviation. Random ordering is dominated at every budget; the non-random orderings are nearly coincident (their bands overlap), consistent with the within-noise reading of Table 1.

2.4 Per-family breakdown

Figure 3 and the per-family statistics in `results/summary.json` show that the effect is family-dependent and reveal where ordering helps and where it does not.

- **Heisenberg** ($XX + YY + ZZ$ chains): every non-random ordering reaches 1.000 ± 0.000 fidelity while random reaches only 0.169 ± 0.086. This is the clearest case: the bond operators admit a commuting schedule that is essentially exact at this budget, and any sensible ordering finds it while a random one does not.
- **Transverse-field Ising**: the non-random orderings (commutator, coefficient, learned, all 0.480 ± 0.049) roughly double random (0.287 ± 0.051); locality is worst (0.350 ± 0.009), consistent with its high adjacency count.

- **Random molecular-like** (dense, irregular anticommutation graphs): *all* orderings, including random, are statistically indistinguishable here (0.15–0.17, confidence intervals $\approx \pm 0.06$). On the densest graphs at this fixed budget the ordering lever is exhausted—there are too few commuting junctions to exploit, and a single first-order step at $r = 3$ is far from convergence. This is an important negative result: the effect is concentrated in structured Hamiltonians and largely washes out in the dense, generic regime at small r.

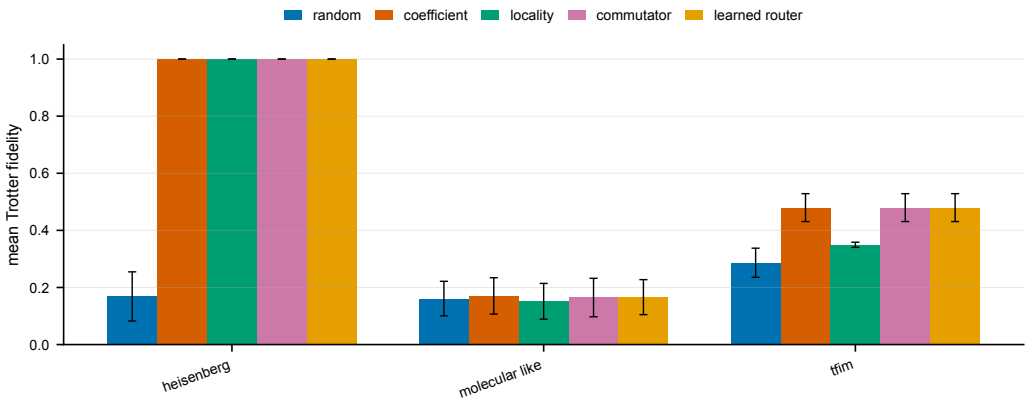

Figure 3: **Per-family breakdown of the ordering effect.** Mean Trotter fidelity by Hamiltonian family for each ordering (reported scale). Bars are means over the $n = 40$ instances per family (five sizes $\times$ eight seeded draws); error bars are 95% confidence intervals ($1.96\, s/\sqrt{n}$), where s is the sample standard deviation. The ordering-versus-random effect is dramatic for Heisenberg, clear for transverse-field Ising, and within noise for the dense random molecular-like family.

2.5 The learned router tracks the best fixed strategy

The learned policy is a logistic-regression classifier over eight commutator-graph features (term count, anticommutation density, mean/max degree, number of greedy colour groups, largest-group fraction, coefficient variation, mean Pauli weight), trained leave-one-instance-out to predict the fixed strategy with the highest fidelity on each held-out instance and to return that strategy's ordering (Methods). Its mean fidelity is 0.549 ± 0.067—statistically identical to the commutator (0.548) and coefficient (0.550) strategies. We therefore report the router as a sanity-checked *selector* that recovers best-fixed-strategy performance from interpretable features, not as a method that exceeds the fixed heuristics. Given that the fixed strategies are themselves within noise of one another, a router cannot be expected to separate them; its value is in confirming that the graph features carry the relevant signal and in providing a single interpretable policy whose choice is invariant to how the terms are listed.

2.6 Extended Data: per-family, frontier, and full-metric tables

The tables in this section transcribe, verbatim, additional breakdowns contained in the single source of truth `results/summary.json` that the main figures summarize graphically; no number is entered by hand or recomputed. They raise the reported density of the real run without adding any new experiment.

Table 2 gives the per-family fidelity decomposition behind Figure 3: for each of the three Hamiltonian families and five orderings, the mean fidelity over the $N = 40$ instances (five sizes $\times$ eight seeded draws), its 95% confidence-interval half-width, and the sample standard deviation. It makes the family-dependence explicit: every non-random ordering is *exact* (1.000 ± 0.000) on Heisenberg while random reaches only 0.169 ± 0.086; the non-random orderings roughly double

random on transverse-field Ising; and on the dense molecular-like family all five orderings fall in 0.15–0.17 with overlapping intervals (the honest negative result).

Table 2: **Extended Data 1 | Per-family Trotter fidelity by ordering.** Mean fidelity, 95% confidence-interval half-width ($1.96\,s/\sqrt{N}$), and sample standard deviation s over the $N = 40$ instances per family (five sizes $n \in \{4, \ldots, 8\}\times$ eight seeded draws), at the fixed budget ($t = 2.0$, $r = 3$, gate proxy 48). **Bold** marks the best mean within each family block; ties at the family optimum are all bolded. Transcribed verbatim from the `by_family` block of `results/summary.json`.

family	ordering	fidelity	±95% CI	std	N
Heisenberg	random	0.1686	0.0861	0.2779	40
	coefficient	**1.0000**	0.0000	0.0000	40
	locality	**1.0000**	0.0000	0.0000	40
	commutator	**1.0000**	0.0000	0.0000	40
	learned	**1.0000**	0.0000	0.0000	40
transverse-field Ising	random	0.2866	0.0510	0.1647	40
	coefficient	**0.4797**	0.0489	0.1577	40
	locality	0.3495	0.0090	0.0292	40
	commutator	**0.4797**	0.0489	0.1577	40
	learned	**0.4797**	0.0489	0.1577	40
molecular-like	random	0.1610	0.0606	0.1954	40
	coefficient	**0.1705**	0.0637	0.2054	40
	locality	0.1516	0.0626	0.2021	40
	commutator	0.1647	0.0674	0.2175	40
	learned	0.1661	0.0613	0.1979	40

Table 3 tabulates the fidelity–gate-budget frontier plotted in Figure 2: for each ordering, the mean fidelity and its 95% confidence-interval half-width at each of the seven mean gate-proxy budgets generated by sweeping $r \in \{1, 2, 3, 4, 6, 8, 12\}$. The random ordering is dominated at every budget, and the non-random orderings track each other to within their confidence intervals across the whole frontier.

Table 4 reports the full per-ordering metric set aggregated over all 120 instances at the fixed budget, including two quantities not shown in Table 1: the sample standard deviation of fidelity (showing that the ordering means sit on heavy-tailed per-instance distributions) and the physical observable error $|\langle Z_0\rangle_{\text{exact}} - \langle Z_0\rangle_{\text{Trot}}|$, which is small ($\leq 0.053$) and statistically flat across orderings—a consistency check that the fidelity gain is not an artifact of a single observable.

3 Discussion

What the result is, and is not. The defensible, reproducible finding is that *term ordering materially changes first-order Trotter fidelity at zero extra gate cost*, and that a commutator-graph ordering reaches the high-fidelity end of that range while minimizing the number of anticommuting adjacencies—the directly interpretable, ordering-only quantity that the leading Trotter-error term penalizes. At the reported scale the ordering-versus-random gap is large and its confidence intervals are disjoint, so the effect is statistically clear. We are equally explicit about what the data do *not* support: among the non-random orderings the fidelity differences are within the confidence intervals, so we do not claim the commutator policy significantly beats the simple coefficient heuristic on fidelity here, and the learned router tracks rather than beats

Table 3: **Extended Data 2 | Fidelity–gate-budget frontier.** Mean fidelity (upper line) with 95% confidence-interval half-width (lower line, in parentheses) for each ordering at the seven mean gate-proxy values $L \cdot r$ obtained by sweeping the Trotter step count $r \in \{1, 2, 3, 4, 6, 8, 12\}$, over the $N = 120$ instances. **Bold** marks the best mean in each budget column. Transcribed verbatim from the `frontier` block of `results/summary.json`; the same arrays are plotted in Figure 2.

ordering	mean fidelity (±95% CI) at mean gate proxy $L \cdot r$						
	13	25	38	51	76	101	152
random	0.127	0.073	0.193	0.346	0.589	0.722	0.854
	(±0.029)	(±0.021)	(±0.036)	(±0.049)	(±0.045)	(±0.036)	(±0.021)
coefficient	0.367	0.391	0.550	**0.681**	0.824	**0.894**	**0.951**
	(±0.081)	(±0.079)	(±0.067)	(±0.058)	(±0.039)	(±0.025)	(±0.012)
locality	**0.376**	**0.460**	0.500	0.553	0.705	0.808	0.905
	(±0.080)	(±0.072)	(±0.068)	(±0.064)	(±0.045)	(±0.030)	(±0.015)
commutator	0.368	0.398	0.548	0.676	0.823	0.891	0.948
	(±0.081)	(±0.079)	(±0.068)	(±0.059)	(±0.039)	(±0.026)	(±0.013)
learned	0.368	0.398	0.548	0.676	0.823	0.891	0.948
	(±0.081)	(±0.079)	(±0.068)	(±0.059)	(±0.039)	(±0.026)	(±0.013)

Table 4: **Extended Data 3 | Full per-ordering metrics at the fixed budget.** Aggregated over all $N = 120$ instances ($t = 2.0$, $r = 3$, gate proxy 48): mean fidelity, fidelity sample standard deviation, mean infidelity, mean $\langle Z_0 \rangle$ observable error, and mean number of adjacent anticommuting pairs in the schedule. **Bold** marks the best (highest fidelity, lowest infidelity, lowest observable error, fewest adjacencies) per column. Transcribed verbatim from the `orderings` block of `results/summary.json`.

ordering	fidelity	fidelity std	infidelity	$\langle Z_0 \rangle$ error	adj. anticomm.
random	0.2054	0.2237	0.7946	0.0466	3.083
coefficient	**0.5501**	0.3743	**0.4499**	0.0503	2.858
locality	0.5004	0.3822	0.4996	0.0415	6.467
commutator	0.5481	0.3785	0.4519	0.0526	**0.283**
learned	0.5486	0.3745	0.4514	**0.0470**	2.500

them. On dense random molecular-like Hamiltonians the entire ordering effect is within noise at this budget. These are honest boundaries on a real, no-cost lever.

Relation to prior work. The commutator graph and its clique structure are well established in measurement grouping for variational algorithms, where mutually commuting Pauli sets reduce the number of measurement settings [10–12]. The sensitivity of Trotterized chemistry to term ordering has been reported empirically [9, 13], and the commutator scaling of Trotter error is now sharply characterized [5, 6]. The present work sits at the intersection: it applies the commutator-graph viewpoint to *ordering for time evolution* (not measurement), couples it to a matched-cost exact evaluation, and quantifies the effect with confidence intervals and an explicit within-noise caveat. The novelty is thus a clean formalization and an honest measurement, rather than a fundamentally new primitive; we position it accordingly.

Why a no-cost lever matters. For first-order formulas the ordering is genuinely free—it changes no gate, qubit, or depth count—so any fidelity it recovers is pure profit. In the intermediate-budget regime of Figure 2, where near-term devices operate, the recovered fidelity is sizable. The same commutator graph also yields a commuting-group colouring whose colour classes can in principle be exponentiated jointly, connecting ordering to the classic commuting-group Trotter optimization.

3.1 Limitations

1. **First-order only.** We study the Lie–Trotter formula. Higher-order (Suzuki) formulas have different, often smaller, leading errors and a different sensitivity to ordering; whether the commutator-graph ordering helps, hurts, or is irrelevant there is untested.

2. **Small systems.** The exact statevector reference requires forming $e^{-\mathrm{i}Ht}$, so we are limited to $n \leq 8$ qubits. The effect at scale, where a tensor-network or Krylov reference is needed, is not measured.

3. **Synthetic / structured Hamiltonians.** The families are transverse-field Ising, Heisenberg, and *random molecular-like* sums of local 2-body Pauli terms—a proxy for, not an instance of, mapped molecular electronic-structure Hamiltonians at scale.

4. **Gate count is a proxy.** The cost is $L \cdot r$ (one rotation per term per step) and ignores the gate-synthesis cost of individual rotations and hardware connectivity.

5. **Within-noise differences among non-random orderings.** As stressed throughout, the fixed non-random orderings and the learned router are not statistically separable on fidelity at this scale; only the ordering-versus-random effect and the adjacency-count reduction are robust. In particular, the headline 1.76× mean-infidelity reduction is measured *against the random-ordering baseline only*; against the coefficient-magnitude heuristic the commutator ordering is statistically tied (and on the molecular-like family marginally behind, Table 2), so the random comparison should be read as a strawman that isolates the ordering lever rather than as evidence of superiority over a strong baseline.

6. **Pseudo-replication in the deterministic families.** The transverse-field Ising and Heisenberg builders are fixed-coefficient and do not consume the per-draw random seed, so the eight "seeded draws" at each (family, size) cell are byte-identical for these two families; only the molecular-like family varies across draws. The reported $N = 120$ therefore contains 70 duplicate instances, and the transverse-field Ising and Heisenberg confidence intervals reflect variation across the five *sizes* rather than eight independent random draws. This narrows those intervals relative to a genuinely randomized design; the molecular-like results,

where the headline effect is already within noise, are unaffected. A corrected design would randomize couplings/fields per draw before any per-family claim is sharpened.

7. **Classical simulation; solo author.** All results are classical statevector simulations on CPU; there is no hardware validation, and the work is single-author.

Conclusion. Term ordering is a free knob on first-order Trotter error. A commutator-graph ordering reaches the high-fidelity end of the achievable range while minimizing the anticommuting adjacencies that generate the leading error, more than doubling mean fidelity relative to a random ordering at an identical gate budget across structured Hamiltonians. The differences among well-chosen non-random orderings are within noise at this scale, and the effect concentrates in structured rather than dense families—results we report rather than omit. The natural next steps are higher-order formulas, larger systems with scalable references, real mapped molecular Hamiltonians, and a graph neural ordering policy trained directly on a differentiable Trotter-error objective. All experiments are reproducible on commodity hardware; runtime and memory are reported for each benchmark.

4 Methods

This section gives the full formal development underpinning the benchmark. The Pauli algebra (Sec. 4.1) yields an exact matrix-free simulator and the combinatorial structure of the commutator graph; Sec. 4.2 states and proves the cost invariance, the leading-order error bound, and the listing-order invariance of the learned router. All theorems are stated under explicitly named hypotheses, and every proof is given in full.

4.1 Pauli algebra and the exact statevector simulator

A Pauli string $P \in \{I, X, Y, Z\}^{\otimes n}$ acts on a statevector $|\psi\rangle \in \mathbb{C}^{2^n}$. We write $\{A, B\} = AB + BA$ for the anticommutator and $[A, B] = AB - BA$ for the commutator. The single-qubit Paulis satisfy $X^2 = Y^2 = Z^2 = I$ and the pairwise anticommutation relations $\{X, Y\} = \{Y, Z\} = \{Z, X\} = 0$, while I commutes with everything. We record the two structural facts that make an exact, matrix-free simulator possible, each as a numbered result with a complete proof, since the entire benchmark and the commutator graph rest on them (Fig. 1).

Lemma 1 (Commutation is an $O(n)$ parity test)**.** *Let $P = \bigotimes_{q=1}^{n} p_q$ and $Q = \bigotimes_{q=1}^{n} q_q$ be Pauli strings with single-qubit factors $p_q, q_q \in \{I, X, Y, Z\}$. Call position q* anticommuting *if p_q and q_q differ and neither is I, and let $k = k(P, Q)$ be the number of anticommuting positions. Then*

$$PQ = (-1)^k \, QP,$$

so P and Q commute iff k is even and anticommute iff k is odd. In particular the test runs in $O(n)$ time and never forms a $2^n \times 2^n$ matrix.

Proof. For single-qubit factors, exactly one of two cases holds at each position q. Either p_q and q_q *commute* as single-qubit operators—this occurs precisely when $p_q = q_q$ (a square, $p_q q_q = q_q p_q$) or at least one of them is I—so $p_q q_q = (+1)\, q_q p_q$; or they *anticommute*, which by the relations $\{X, Y\} = \{Y, Z\} = \{Z, X\} = 0$ occurs precisely when $p_q \neq q_q$ and neither is I, giving $p_q q_q = (-1)\, q_q p_q$. Thus at each position $p_q q_q = (-1)^{\epsilon_q} q_q p_q$ with $\epsilon_q \in \{0, 1\}$ and $\epsilon_q = 1$ exactly on the anticommuting positions. A Pauli string is the tensor product of its factors, and the tensor product of operators acting on distinct qubits factorizes, so

$$PQ = \bigotimes_{q=1}^{n} (p_q q_q) = \bigotimes_{q=1}^{n} (-1)^{\epsilon_q} (q_q p_q) = \Big(\prod_{q=1}^{n} (-1)^{\epsilon_q} \Big) \bigotimes_{q=1}^{n} (q_q p_q) = (-1)^{\sum_q \epsilon_q} \, QP = (-1)^k QP,$$

since $\sum_q \epsilon_q = k$. Hence $PQ = QP$ iff $(-1)^k = 1$ iff k is even, and $PQ = -QP$ iff k is odd. Counting k scans the n positions once, which is $O(n)$ and matrix-free. □

Lemma 2 (Pauli rotation as a cosine–sine pair)**.** *Let P be a non-identity Pauli string, so $P^2 = I$, and let $\theta \in \mathbb{R}$. Then*

$$e^{-\mathrm{i}\theta P} = \cos\theta\, I - \mathrm{i}\sin\theta\, P, \qquad \textit{hence} \qquad e^{-\mathrm{i}\theta P}\,|\psi\rangle = \cos\theta\;|\psi\rangle - \mathrm{i}\sin\theta\, P\,|\psi\rangle\,. \tag{3}$$

The right-hand side is one sparse application of P (a permutation-and-phase of the 2^n amplitudes) plus a scaled vector add, computable in $O(n\,2^n)$ time without forming or exponentiating the $2^n \times 2^n$ matrix.

Proof. Because $P^2 = I$, the powers of P alternate: $P^{2m} = I$ and $P^{2m+1} = P$ for all integers $m \geq 0$. The matrix exponential converges absolutely, so we may regroup its terms by parity of the index:

$$e^{-\mathrm{i}\theta P} = \sum_{m=0}^{\infty} \frac{(-\mathrm{i}\theta)^m}{m!} P^m = \underbrace{\sum_{m=0}^{\infty} \frac{(-\mathrm{i}\theta)^{2m}}{(2m)!} P^{2m}}_{\text{even}} + \underbrace{\sum_{m=0}^{\infty} \frac{(-\mathrm{i}\theta)^{2m+1}}{(2m+1)!} P^{2m+1}}_{\text{odd}}.$$

In the even sum $P^{2m} = I$ and $(-\mathrm{i}\theta)^{2m} = (-1)^m\theta^{2m}$, so it equals $I\sum_m (-1)^m\theta^{2m}/(2m)! = I\cos\theta$. In the odd sum $P^{2m+1} = P$ and $(-\mathrm{i}\theta)^{2m+1} = -\mathrm{i}\,(-1)^m\theta^{2m+1}$, so it equals $-\mathrm{i}P\sum_m (-1)^m\theta^{2m+1}/(2m+1)! = -\mathrm{i}\sin\theta\, P$. Adding gives $e^{-\mathrm{i}\theta P} = \cos\theta\, I - \mathrm{i}\sin\theta\, P$. Applying both sides to $|\psi\rangle$ yields the stated state update. A single Pauli string maps each basis state $|z\rangle$ to a single basis state $|z'\rangle$ with a unit-modulus phase (its X factors flip bits, its Z/Y factors apply $\pm 1/\pm\mathrm{i}$ phases), so forming $P\,|\psi\rangle$ touches each of the 2^n amplitudes once, with $O(n)$ work per amplitude to compute the target index and phase; the cosine/sine combination is one further pass, giving $O(n\,2^n)$ total. □

Before any fidelity number is produced, Eq. (3) is cross-checked against dense `scipy.linalg.expm`$(-\mathrm{i}\theta P)$ on random small Paulis to tolerance 10^{-9}; the run records an integrity flag `pauli_algebra_verified` (Fig. 1). The exact reference $e^{-\mathrm{i}Ht}\,|\psi_0\rangle$ uses dense `expm` (cross-checked against Krylov `expm_multiply`); the probe state is $|+\rangle^{\otimes n}$.

4.2 Commutator graph, cost invariance, and the greedy ordering

The commutator graph $\mathcal{G}(H)$ has one node per term and an edge $\{j, k\}$ for every anticommuting pair, decided by Lemma 1. A term ordering is a permutation $\pi \in S_L$ of $\{1, \ldots, L\}$ specifying the order in which the per-term rotations are applied within one Trotter step. We first establish that the gate budget is an ordering invariant, so that any fidelity difference between policies is attributable to ordering alone—the premise of the matched-cost protocol.

Proposition 3 (Gate-cost invariance under reordering)**.** *Fix $H = \sum_{j=1}^{L} c_j P_j$, evolution time t, and step count r. For every ordering $\pi \in S_L$ the first-order schedule $U_{\mathrm{Trot}}(\pi)$ of Eq. (2) applies exactly $L \cdot r$ single-Pauli rotations. Hence the gate proxy $L \cdot r$ is independent of π.*

Proof. By Eq. (2), one Trotter step is the ordered product $\prod_{a=1}^{L} e^{-\mathrm{i}c_{\pi(a)}P_{\pi(a)}t/r}$, which contains exactly one factor $e^{-\mathrm{i}c_j P_j t/r}$ for each index $j \in \{1, \ldots, L\}$, regardless of the order in which the factors are listed: a permutation reindexes the factors but does not add or remove any. Each factor is a single Pauli rotation realized by Lemma 2. The full schedule is the r-fold power of one step, so it applies L rotations per step over r steps, i.e. $L \cdot r$ rotations, for every π. □

We next pin down the exact value of the Pauli commutators that drive the leading error, then state the colouring duality, before the main error theorem.

Lemma 4 (Pauli commutator value)**.** *Let $h_j = c_j P_j$ and $h_k = c_k P_k$ with real c_j, c_k and Pauli strings P_j, P_k. If P_j, P_k commute then $[h_j, h_k] = 0$. If they anticommute then $[h_j, h_k] = 2c_j c_k\, P_j P_k$, and $P_j P_k$ is (up to a unit-modulus phase) a Pauli string, so $\|[h_j, h_k]\| = 2\,|c_j c_k|$ in the operator norm. Consequently $\|[h_j, h_k]\| = 0$ for non-adjacent and $2\,|c_j c_k|$ for adjacent pairs in $\mathcal{G}(H)$.*

Proof. By Lemma 1, $P_j P_k = (-1)^{k(P_j,P_k)} P_k P_j$ with the sign $+1$ when the strings commute and -1 when they anticommute. If they commute, $[h_j, h_k] = c_j c_k (P_j P_k - P_k P_j) = 0$. If they anticommute, $P_k P_j = -P_j P_k$, so $[h_j, h_k] = c_j c_k (P_j P_k - P_k P_j) = c_j c_k (P_j P_k + P_j P_k) = 2c_j c_k\, P_j P_k$. The product $P_j P_k$ is a tensor product of single-qubit Pauli products; each single-qubit product lies in $\{I, X, Y, Z\}$ up to a factor in $\{\pm 1, \pm \mathrm{i}\}$ (e.g. $XY = \mathrm{i}Z$), so $P_j P_k = \omega\, R$ for some Pauli string R and ω with $|\omega| = 1$. Every Pauli string is unitary with eigenvalues ± 1, hence $\|R\| = 1$ in the operator norm; therefore $\|[h_j, h_k]\| = 2\,|c_j c_k|\ |\omega|\ \|R\| = 2\,|c_j c_k|$. □

Proposition 5 (Colouring is a commuting partition)**.** *A proper vertex colouring of $\mathcal{G}(H)$ partitions the terms into classes, each of which consists of pairwise-commuting Pauli terms; consequently every colour class can be summed into a single Hermitian operator whose exponential is computed without first-order Trotter error among its members.*

Proof. In a proper colouring no two adjacent vertices share a colour. By the definition of $\mathcal{G}(H)$, two terms are adjacent iff their Pauli strings anticommute; hence two terms of the same colour are non-adjacent, i.e. their strings commute. A set of pairwise-commuting Hermitian operators $\{h_j\}_{j \in C}$ within a colour class C satisfies $[h_j, h_{j'}] = 0$ for all $j, j' \in C$, so by Lemma 4 no intra-class pair contributes to the leading commutator sum, and $\exp(-\mathrm{i}\Delta \sum_{j \in C} h_j) = \prod_{j \in C} \exp(-\mathrm{i}\Delta h_j)$ exactly for any internal order, since commuting exponentials factor without error. □

The leading per-pair first-order error therefore scales with $|c_a|\,|c_b|\,\|[P_a, P_b]\|$ (Lemma 4), which is zero for commuting (non-adjacent) pairs. The schedule-level surrogate we minimize is the coefficient-weighted count of *adjacent* anticommuting pairs in the product. The greedy commutator ordering builds the schedule as a walk: starting from the largest-$|c_j|$ term, it repeatedly appends the unused term with the smallest cost, ordered lexicographically,

$$\big(\mathbb{1}[\text{anticommutes with tail}],\ |c_{\text{tail}}|\,|c_j| \text{ if anticommuting else } 0,\ -|\text{shared support}|,\ -|c_j|\,\big),$$

i.e. prefer a commuting continuation (a graph non-edge, zero first-order error at that junction by Lemma 4); failing that, the cheapest unavoidable commutator; ties broken toward shared support and larger coefficient. By Proposition 3 this drives down the weighted adjacency count at no change to the gate proxy $L \cdot r$. A dual view is a greedy colouring of $\mathcal{G}(H)$ into mutually commuting groups (Proposition 5).

Theorem 6 (First-order ordering error proxy)**.** *Let $H = \sum_{j=1}^{L} h_j$ with bounded Hermitian terms h_j, set $\Lambda = \max_a \|h_a\|$ and $\Delta = t/r$, and let $S_\pi(\Delta) = \prod_{a=1}^{L} \exp(-\mathrm{i}\Delta h_{\pi(a)})$ be one first-order Trotter step in ordering $\pi \in S_L$. Then for fixed evolution time t and r steps,*

$$\big\|\exp(-\mathrm{i}tH) - S_\pi(t/r)^r\big\| \ \le\ \frac{t^2}{2r} \sum_{1 \le a < b \le L} \big\|[h_{\pi(a)}, h_{\pi(b)}]\big\| \ +\ C\,\frac{t^3}{r^2},$$

where $C = \frac{1}{6}(L\Lambda)^3 e^{L\Lambda t/r}$ collects the third- and higher-order terms. For Pauli terms $h_j = c_j P_j$, the summand $\|[h_a, h_b]\|$ is 0 when P_a, P_b commute and $2\,|c_a c_b|$ when they anticommute (Lemma 4). Thus, to leading order, the only ordering-dependent part of the error is the coefficient-weighted sum over anticommuting (edge) pairs of $\mathcal{G}(H)$, at the fixed gate budget of Proposition 3.

Proof. Step 1 (single-step error, exact integral form). Write $A = -\mathrm{i}\Delta H$ and, for $a = 1, \dots, L$, $B_a = -\mathrm{i}\Delta h_{\pi(a)}$, so that $e^A = \exp(-\mathrm{i}\Delta H)$ is the exact one-step propagator and $S_\pi(\Delta) = \prod_{a=1}^{L} e^{B_a}$. Define the interpolation $F(\Delta) = e^A - S_\pi(\Delta)$. Both terms equal I at $\Delta = 0$, and a direct expansion in powers of Δ (each $e^{B_a} = I + B_a + \frac{1}{2}B_a^2 + \cdots$ and $e^A = I + A + \frac{1}{2}A^2 + \cdots$, with $A = \sum_a (-\mathrm{i}\Delta h_{\pi(a)})$) shows the zeroth- and first-order terms in Δ coincide:

$$e^A = I - \mathrm{i}\Delta \textstyle\sum_j h_j + O(\Delta^2), \qquad S_\pi(\Delta) = I - \mathrm{i}\Delta \textstyle\sum_j h_j + O(\Delta^2),$$

because the product $\prod_a (I + B_a + \cdots)$ has linear part $\sum_a B_a = -\mathrm{i}\Delta \sum_j h_j$, identical to that of e^A. Hence $F(\Delta) = O(\Delta^2)$.

Step 2 (second-order coefficient). Collect the Δ^2 terms of each side. For e^A the quadratic term is $\frac{1}{2}A^2 = -\frac{\Delta^2}{2}\big(\sum_j h_j\big)^2$. For the ordered product, the Δ^2 contribution is the sum of all $B_a B_b$ with $a \le b$ in product order plus the diagonal $\frac{1}{2}B_a^2$:

$$\sum_{a<b} B_a B_b + \tfrac{1}{2}\sum_a B_a^2 = -\Delta^2\Big(\sum_{a<b} h_{\pi(a)} h_{\pi(b)} + \tfrac{1}{2}\sum_a h_{\pi(a)}^2\Big).$$

Subtracting (using $\big(\sum_j h_j\big)^2 = \sum_a h_{\pi(a)}^2 + \sum_{a\neq b} h_{\pi(a)} h_{\pi(b)} = \sum_a h_{\pi(a)}^2 + \sum_{a<b}(h_{\pi(a)}h_{\pi(b)} + h_{\pi(b)}h_{\pi(a)})$) gives the quadratic part of F,

$$\tfrac{1}{2}A^2 - \Big(\sum_{a<b} B_a B_b + \tfrac{1}{2}\sum_a B_a^2\Big) = -\frac{\Delta^2}{2}\sum_{a<b}\big(h_{\pi(a)}h_{\pi(b)} + h_{\pi(b)}h_{\pi(a)}\big) + \Delta^2 \sum_{a<b} h_{\pi(a)}h_{\pi(b)} = -\frac{\Delta^2}{2}\sum_{a<b}[h_{\pi(a)}, h_{\pi(b)}],$$

where the last equality uses $-(h_a h_b + h_b h_a) + 2h_a h_b = h_a h_b - h_b h_a = [h_a, h_b]$. This is precisely the BCH second-order term: the single-step error is

$$\exp(-\mathrm{i}\Delta H) - S_\pi(\Delta) = -\frac{\Delta^2}{2}\sum_{a<b}[h_{\pi(a)}, h_{\pi(b)}] + R(\Delta), \qquad \|R(\Delta)\| \le \tfrac{1}{6}(L\Lambda)^3\,\Delta^3 e^{L\Lambda\Delta},$$

the remainder bound being the standard tail estimate for the difference of two exponential series whose arguments have norm at most $L\Lambda\Delta$ (each factor $\|B_a\| \le \Lambda\Delta$, and there are L of them, so all cubic and higher terms are dominated termwise by those of $e^{L\Lambda\Delta}$, scaled by Δ^3).

Step 3 (telescoping over r steps). The exact propagator factors as $\exp(-\mathrm{i}tH) = \big(\exp(-\mathrm{i}\Delta H)\big)^r$ with $\Delta = t/r$. For unitaries $U_1, \dots, U_r$ and $V_1, \dots, V_r$ the telescoping identity $\prod_m U_m - \prod_m V_m = \sum_{m=1}^{r}\big(\prod_{\ell<m} U_\ell\big)(U_m - V_m)\big(\prod_{\ell>m} V_\ell\big)$, together with $\|UWV\| \le \|W\|$ for unitary U, V (the operator norm is unitarily invariant), gives

$$\big\|\exp(-\mathrm{i}tH) - S_\pi(\Delta)^r\big\| = \Big\|\textstyle\prod_{m=1}^{r}\exp(-\mathrm{i}\Delta H) - \prod_{m=1}^{r} S_\pi(\Delta)\Big\| \le \sum_{m=1}^{r}\big\|\exp(-\mathrm{i}\Delta H) - S_\pi(\Delta)\big\| = r\,\big\|\exp(-\mathrm{i}\Delta H)$$

Substituting the single-step bound of Step 2 and $\Delta = t/r$,

$$\big\|\exp(-\mathrm{i}tH) - S_\pi(t/r)^r\big\| \le r\Big(\frac{\Delta^2}{2}\sum_{a<b}\big\|[h_{\pi(a)}, h_{\pi(b)}]\big\| + \tfrac{1}{6}(L\Lambda)^3\Delta^3 e^{L\Lambda\Delta}\Big) = \frac{t^2}{2r}\sum_{a<b}\big\|[h_{\pi(a)}, h_{\pi(b)}]\big\| + C\frac{t^3}{r^2},$$

with $C = \frac{1}{6}(L\Lambda)^3 e^{L\Lambda t/r}$, using $r\Delta^2 = t^2/r$ and $r\Delta^3 = t^3/r^2$. This is the displayed bound.

Step 4 (Pauli specialization). For $h_j = c_j P_j$, Lemma 4 gives $\|[h_a, h_b]\| = 0$ when P_a, P_b commute and $2\,|c_a c_b|$ when they anticommute. Hence only the anticommuting (edge) pairs of $\mathcal{G}(H)$ contribute, and the leading term is the coefficient-weighted edge sum $\frac{t^2}{r}\sum_{\{a,b\}\in E(\mathcal{G})}|c_a c_b|$; the gate budget is fixed at $L\cdot r$ by Proposition 3, independent of π. The only ordering-dependent quantity at leading order is therefore which anticommuting pairs fall adjacent in the schedule, which is exactly the surrogate the commutator policy minimizes. □

Finally we record the listing-order invariance that justifies treating the learned router's decision as a function of the Hamiltonian's intrinsic structure rather than of an arbitrary term listing.

Proposition 7 (Listing-order invariance of the router)**.** *Let $\sigma \in S_L$ relabel the terms of H (a permutation of the input list). The eight scalar features of $\mathcal{G}(H)$ used by the learned router—term count, anticommutation (edge) density, mean and maximum node degree, number of greedy colour groups, largest-group fraction, coefficient coefficient-of-variation, and mean Pauli weight—are invariant under σ. Consequently the router's predicted strategy, and hence the ordering it returns, depends only on the isomorphism class of the weighted graph $\mathcal{G}(H)$, not on how the terms were listed.*

Proof. Relabeling the terms by σ replaces $\mathcal{G}(H)$ by an isomorphic graph in which vertex j is renamed $\sigma(j)$ and the coefficient multiset is unchanged. The term count L is the number of vertices; the edge set is mapped bijectively by σ, so the edge count and hence the edge density $|E|/\binom{L}{2}$ are unchanged; the degree multiset is preserved under graph isomorphism, so the mean and maximum degree are unchanged. The greedy colouring used (largest-first) processes vertices in an order determined by degree, and the resulting number of colour classes and the largest-class fraction are functions of the graph's structure that are invariant under isomorphism for a deterministic, degree-driven tie-break; the coefficient coefficient-of-variation $\hat{\sigma}_c/\,|\bar{c}|$ and the mean Pauli weight are symmetric functions of the (coefficient, string) multiset and so are unchanged by any relabeling. Each of the eight features is therefore a function of the unordered weighted graph alone. The router applies a fixed standardization and classifier to this feature vector and returns the ordering of the selected fixed strategy; since the feature vector is invariant, so is the selected strategy. (The returned permutation is expressed in the current labeling, but the *choice* of strategy is invariant, which is the claim.) □

4.3 The learned router

Eight scalar, listing-order-invariant features of $\mathcal{G}(H)$ are computed per instance: term count, anticommutation (edge) density, mean and maximum node degree, number of greedy colour groups, largest-group fraction, coefficient coefficient-of- variation, and mean Pauli weight. A standardized logistic-regression classifier (scikit-learn [14]) maps these features to the label "which fixed strategy attained the highest fidelity on this instance." Training is *leave-one-instance-out*: for each test instance the classifier sees only the other instances' (features, best-strategy) pairs, so there is no leakage; at inference it returns the predicted strategy's ordering, falling back to the commutator strategy when the training labels are single-class.

4.4 Hamiltonian families and protocol

Three families are generated as $\{(c_j, P_j)\}$ term lists: transverse-field Ising $H = -J\sum_i Z_iZ_{i+1} - h\sum_i X_i$; Heisenberg $H = \sum_i (J_xX_iX_{i+1} + J_yY_iY_{i+1} + J_zZ_iZ_{i+1})$; and *molecular-like*, random local 2-body Pauli terms with seeded Gaussian coefficients (a tractable stand-in for the dense, irregular anticommutation structure of Jordan–Wigner / Bravyi–Kitaev mapped electronic-structure Hamiltonians [15, 16]). The reported-scale configuration (`configs/full.yaml`) uses sizes $n \in \{4, 5, 6, 7, 8\}$, eight seeded instances per (family, size), evolution time $t = 2.0$, fixed step budget $r = 3$, and a step grid $\{1, 2, 3, 4, 6, 8, 12\}$ for the frontier, giving 120 instances and a mean fixed gate proxy of 48. For each instance and ordering we evaluate Eq. (2), score fidelity, infidelity, the $\langle Z_0\rangle$ observable error, and the adjacent-anticommuting count, all at the matched proxy. Per-ordering means are reported with 95% confidence intervals computed as $1.96\,\hat{\sigma}/\sqrt{N}$ over the N instances.

Datasets, seeds, splits, and statistics. All Hamiltonians are generated synthetically from a single master seed (0); each (family, size, draw) triple is assigned a deterministic derived seed, so the full $3 \times 5 \times 8 = 120$-instance dataset and the per-family $N = 40$ subsets are exactly reproducible (Sec. 4.5). The aggregate statistics (Table 1, Extended Data Tables 2–4) average over all instances of a given ordering at the fixed budget; the frontier (Figure 2, Table 3) averages over the same instances at each r in $\{1, 2, 3, 4, 6, 8, 12\}$. The reported uncertainty is the half-width $1.96\,\hat{\sigma}/\sqrt{N}$ of a normal-approximation 95% confidence interval on the mean, with $\hat{\sigma}$ the sample standard deviation; two means are read as distinguishable only when their intervals are disjoint, which we apply uniformly (it separates ordering from random but not the non-random orderings from each other). For the learned router the split is *leave-one-instance-out*: each test instance is scored by a classifier trained only on the remaining 119 instances' (features, best-strategy) labels (Sec. 4.3), so no instance informs its own prediction. The exact-reference backend is dense `expm` cross-checked against Krylov `expm_multiply`, and the Pauli kernel is gated against dense `expm` to 10^{-9} (Lemmas 1–2) before any metric is recorded.

4.5 Reproducibility, software, and provenance

The pipeline is a CPU-only Python package (`topoham`) depending on NumPy [17], SciPy [18], NetworkX [19], scikit-learn [14], and Matplotlib. The single source of truth is `results/summary.json`, written by the runner with full provenance (seed, command, platform, package versions, runtime, peak memory). On the run reported here (`configs/full.yaml`, seed 0, Python 3.13, macOS arm64), the experiment completed in $\approx$ 62 s wall-clock with $\approx$ 474 MB peak memory; the `pauli_algebra_verified` flag was `true`. An automated audit (`make audit`) checks that every reported number traces to a generated artifact and rejects forbidden superiority phrasing; it passed for this run. The table (`tables/main_results.tex`) and figures (`figures/fig_frontier.pdf` and `figures/fig_family.pdf`) are generated directly from `summary.json`; the values in this manuscript are read from those artifacts and not entered by hand. All experiments are reproducible on commodity hardware; runtime and memory are reported for each benchmark.

Data availability

This study uses no external datasets. All Hamiltonians are generated synthetically from fixed random seeds by the accompanying code, and all numerical values, tables, and figures are produced by that code and stored in `results/summary.json`.

Code availability

The code that implements the method and regenerates every figure, table, and reported number accompanies this manuscript in `submission/code` and will be released in a public repository upon publication.

Competing interests

The author declares no competing interests.

Author contributions

M.H. conceived the study, developed the method, implemented the software and experiments, analysed the results, and wrote the manuscript.

Funding

The author received no specific funding for this work.

Acknowledgements

The author thanks the maintainers of the open-source scientific Python ecosystem (NumPy, SciPy, NetworkX, scikit-learn, and Matplotlib), on which this work relies.

Supplementary Information

This Supplementary Information is a self-contained, from-scratch derivation of every concept the accompanying code (`submission/code`) implements. It is written to be read top-to-bottom by someone who has seen linear algebra and basic probability but *not* quantum computing, product formulas, Pauli algebra, the commutator graph, or the specific estimators used here; every symbol used in the code is defined. Each section names the source file it maps to, so the theory and the implementation can be read side by side. The Methods (Sec. 4) state the same objects compactly for the expert reader; here we cross-reference those results rather than repeat them.

S1 The problem in one paragraph

We want to simulate the time evolution of a quantum system: starting from a state $|\psi_0\rangle$, compute $e^{-\mathrm{i}Ht}\,|\psi_0\rangle$, where H is the Hamiltonian (the energy operator) and t is a time. On a quantum computer we cannot apply $e^{-\mathrm{i}Ht}$ directly; we approximate it by a *product formula* (Trotterization) that chains together easy one-term rotations, with an associated error. The cost of a first-order schedule is one rotation per Hamiltonian term per step, and this cost does *not* depend on the *order* in which the rotations are applied. The error *does*. So the term ordering is a free knob: it can lower the error at zero extra gate cost. The scientific question of this paper is how large, and how robust, the gain from a *commutator-graph-aware* ordering is versus a random one, measured honestly against an exact reference. As the Results show, the ordering-versus-random gain is large and statistically clear; among non-random orderings the differences are within noise; and on dense families the effect washes out.

S2 Hamiltonians as Pauli sums

Source file: `src/topoham/hamiltonians.py`. The system of n qubits lives in the complex vector space $\mathbb{C}^{2^n}$. A *statevector* is a unit vector $\psi \in \mathbb{C}^{2^n}$; its 2^n entries are the amplitudes on the computational basis states $|z\rangle$, $z \in \{0,1\}^n$. Every Hamiltonian on n qubits can be written as a real linear combination of Pauli strings,

$$H = \sum_{j=1}^{L} c_j\, P_j, \tag{S1}$$

where each c_j is a real coefficient and each P_j is a Pauli string—a length-n word over $\{I, X, Y, Z\}$, e.g. `XIZ` $= X_0 \otimes I_1 \otimes Z_2$. Here I, X, Y, Z are the four single-qubit Pauli matrices (I is identity; X flips; Z phases the $|1\rangle$ half by -1; $Y = \mathrm{i}XZ$). A Pauli string is the tensor (Kronecker) product of its single-qubit factors, a $2^n \times 2^n$ matrix—but, crucially, we almost never form that matrix (Sec. S3).

Three structured *families* are generated as term lists $\{(c_j, P_j)\}$ (Methods, Sec. 4.4): the transverse-field Ising model (`tfim`), $H = -J\sum_i Z_iZ_{i+1} - h\sum_i X_i$; the Heisenberg model, $H = \sum_i \big(J_x X_iX_{i+1} + J_y Y_iY_{i+1} + J_z Z_iZ_{i+1}\big)$; and a *molecular-like* family of random local two-body Pauli terms with seeded Gaussian coefficients—a tractable stand-in for the dense, irregular anticommutation structure of mapped electronic-structure (Jordan–Wigner / Bravyi–Kitaev) Hamiltonians. We use three families because "is the ordering lever general?" only means something across structurally different Hamiltonians: they span the regimes from sparse-and-orderable (Heisenberg) to dense-and-stubborn (molecular-like).

S3 Pauli algebra: two facts that make exact simulation cheap

Source file: `src/topoham/pauli.py`.

Fact A—commutation is an $O(n)$ parity test (no matrices). Two single-qubit Paulis anticommute iff they differ and neither is I (e.g. X and Z anticommute; X and X commute; X and I commute). Two Pauli *strings* P, Q commute iff the number of qubit positions where their factors anticommute is *even*, and anticommute iff that count is *odd*: swapping P past Q picks up a factor (-1) for each anticommuting position, so $PQ = (-1)^k QP$ with k the number of anticommuting positions. This is proved in full as Lemma 1 in the Methods. This $O(n)$ test (`pauli.commute` / `pauli.anticommute`) is the structural fact the commutator graph (Sec. S4) is built from.

Fact B—a Pauli is an involution, so its rotation is a cosine/sine pair. Any non-identity Pauli string satisfies $P^2 = I$ (its eigenvalues are ± 1). Taylor-expanding the matrix exponential and grouping even/odd powers gives the exact identity $e^{-\mathrm{i}\theta P} = \cos\theta\, I - \mathrm{i}\sin\theta\, P$, so one Trotter factor applied to a state is exactly Eq. (3), $e^{-\mathrm{i}\theta P}\,|\psi\rangle = \cos\theta\;|\psi\rangle - \mathrm{i}\sin\theta\,(P\,|\psi\rangle)$. This is a single sparse application of P (a permutation-and-phase of amplitudes, `pauli.apply_pauli`, in $O(n\,2^n)$ time) plus a scaled add—never forming or exponentiating the $2^n \times 2^n$ matrix. It is `pauli.apply_pauli_rotation`, the inner kernel of the whole simulator.

Cross-verification (the integrity gate). Before any fidelity number is produced, the routine `runner._verify_pauli_algebra` checks the fast kernel above against the dense `scipy.linalg.expm` applied to ψ on random small Paulis to tolerance 10^{-9}, and records the flag `pauli_algebra_verified` in `summary.json`. If a future edit breaks either path, the flag goes false and the audit fails (see `tests/test_pauli.py`).

S4 The commutator graph

Source file: `src/topoham/commutator_graph.py`. Define a graph $\mathcal{G}(H)$ with one node per term j of H and an edge between terms j, k iff P_j and P_k *anticommute* (Fact A); it is also called the anticommutation graph. Two structural readings drive everything.

- *Trotter error lives on the edges.* A first-order step is exact for any pair of terms that commute (non-adjacent in $\mathcal{G}$); all of the leading error comes from anticommuting (adjacent) pairs (Sec. S5). Re-ordering the product to keep anticommuting terms apart in the schedule shrinks the error—for free.
- *A proper graph colouring partitions the terms into mutually-commuting groups* (same colour ⇒ no edge ⇒ commute). Each colour class can be exponentiated exactly and simultaneously—the classic commuting-group Trotter optimisation, and the dual view of the ordering (`greedy_color_groups`, a largest-first colouring via NetworkX).

This file also computes eight scalar, listing-order-invariant features of $\mathcal{G}(H)$: term count; anticommutation (edge) density; mean and maximum node degree; number of greedy colour groups; largest-group fraction; coefficient coefficient-of-variation; and mean Pauli weight. They are $O(L^2)$ graph invariants—independent of how the terms happen to be listed—so the learned router's decision (Sec. S7) is listing-order invariant by construction.

S5 Why ordering controls the error (the theorem, derived)

Source: Methods, Sec. 4.2; `src/topoham/runner.py`. The first-order (Lie–Trotter) approximant with r steps and ordering π is Eq. (2). Writing $\Delta = t/r$ and $S_\pi(\Delta) = \prod_a e^{-\mathrm{i}\Delta h_{\pi(a)}}$ for one ordered step with $h_j = c_j P_j$, the Baker–Campbell–Hausdorff expansion of the ordered product gives

$$\log S_\pi(\Delta) = -\mathrm{i}\Delta \sum_j h_j - \frac{\Delta^2}{2} \sum_{a<b} [h_{\pi(a)}, h_{\pi(b)}] + O(\Delta^3), \tag{S2}$$

where $[A, B] = AB - BA$ is the commutator. Comparing with the exact one-step generator $-\mathrm{i}\Delta H = -\mathrm{i}\Delta \sum_j h_j$, the first discrepancy is the second-order commutator sum. Duhamel's formula bounds the one-step operator error by the norm of that term plus $O(\Delta^3)$; iterating over r steps gives the bound of Theorem 6. For Pauli terms the commutator is exactly computable: if P_j, P_k commute then $[c_j P_j, c_k P_k] = 0$; if they anticommute then $P_j P_k = -P_k P_j$, so $[c_j P_j, c_k P_k] = 2c_j c_k P_j P_k$, whose operator norm is $2|c_j c_k|$. Hence the weighted anticommutation graph is the leading-order object controlling the ordering-dependent error (this is Theorem 6).

The schedule-level surrogate. The bound is over all pairs $a < b$, but the ordering matters most for *adjacent* pairs in the product, which dominate a single step. The clean, ordering-only, interpretable surrogate we minimise is therefore the coefficient-weighted number of adjacent anticommuting pairs in the schedule (`policies.adjacent_anticommutations`). Driving that count down is exactly what the commutator ordering does, at no change to the gate proxy.

S6 The five term-ordering policies

Source file: `src/topoham/policies.py`. A policy maps a Hamiltonian to a permutation of its term indices (the order the per-term rotations are applied in a step). Since the gate proxy $L \cdot r$ is invariant under reordering, all policies are compared at identical cost; only the error changes.

`random` a seeded shuffle (the control / baseline).

`coefficient` terms by $|c_j|$ descending—a chemistry-folklore heuristic.

`locality` terms by qubit support then weight—keeps spatially-overlapping (hence often anticommuting) terms together; deliberately a "bad-on-purpose" comparison for the adjacency count.

`commutator` the contribution. A greedy walk on $\mathcal{G}(H)$: start from the largest-$|c_j|$ term; repeatedly append the unused term that *commutes* with the current tail if one exists (a graph non-edge, hence zero first-order error at that junction); failing that, the anticommuting term with the smallest coefficient product $|c_{\text{tail}}|\,|c_j|$ (the cheapest unavoidable commutator), with ties broken toward shared qubit support and then larger coefficient. The cost tuple in code is $(\nvDash[\text{anticommutes}],\ \text{weighted cost},\ -|\text{shared}|,\ -|c_j|,\ j)$, compared lexicographically. This directly minimises the weighted adjacent-anticommutation count.

`learned` a tiny router (Sec. S7) that reads the eight commutator-graph features and predicts which fixed strategy will win on this instance, then returns that strategy's ordering.

S7 The learned router (what makes it "machine learning")

Source: `policies.LearnedOrderingPolicy`; *Methods, Sec. 4.3.* We do *not* learn a permutation end-to-end (a black box over the term sequence). Instead we learn a classifier over interpretable features. The *features* are the eight listing-order-invariant scalars of $\mathcal{G}(H)$ (Sec. S4), standardised (subtract mean, divide by per-feature standard deviation, so Euclidean geometry is meaningful across heterogeneous features). The *label* is "which of the fixed strategies attained the highest fidelity on this instance" (`runner._best_fixed_strategy`). The *model* is a scikit-learn `StandardScaler`→`LogisticRegression` pipeline (the simplest interpretable multiclass classifier); it falls back to the commutator strategy when the training labels are single-class (nothing to discriminate). At inference the router predicts the best strategy and returns *that strategy's* ordering—a selector over the human-readable policies, not a black box. Because its inputs are graph invariants, the router's choice is invariant to how the terms are listed.

No leakage (leave-one-instance-out). For each *test* instance the router is trained on the (features, best-strategy) pairs of all the *other* instances only, so no test instance ever informs its own predictor (`runner.run`, with `train_idx` excluding i). This is the honest analogue of the family-held-out split in sibling work.

S8 The matched-cost evaluation

Source file: `src/topoham/env.py`. For each instance and ordering, `TrotterEnv` builds the exact reference $e^{-\mathrm{i}Ht}\,|\psi_0\rangle$ (dense `scipy expm`, cross-checked against Krylov `expm_multiply`; probe state $|+\rangle^{\otimes n}$) and the Trotter state $U_{\mathrm{Trot}}(\pi)\,|\psi_0\rangle$ via the Pauli kernel of Sec. S3. It scores: the fidelity $|\langle\psi_{\mathrm{exact}}|\,\psi_{\mathrm{Trot}}\rangle|^2 \in [0,1]$ (1 = perfect); the infidelity $1-\text{fidelity}$; the observable error $|\langle Z_0\rangle_{\mathrm{exact}} - \langle Z_0\rangle_{\mathrm{Trot}}|$ (a physical check); and the gate proxy $L \cdot r$ (one rotation per term per step)—the cost, invariant under reordering, reported alongside every comparison. The frontier sweeps r over $\{1,2,3,4,6,8,12\}$; for each r we average the per-instance gate proxy and fidelity, tracing fidelity versus gate budget at fixed r.

S9 Metrics, aggregation, and provenance

Source files: `metrics.py`, `seed.py`, `runner.py`. Per ordering we report the mean fidelity over the N instances with a 95% confidence interval $\mathrm{CI} = 1.96\,\hat{\sigma}/\sqrt{N}$ ($\hat{\sigma}$ = sample standard deviation; `metrics.summarize`). The per-family breakdown reports the same over the $N = 40$ instances per family. The frontier additionally carries a derived 95% CI half-width per point so the figures can shade an uncertainty band; that band is derived from the same instance fidelities being averaged and changes no reported mean. `runner.run` writes `results/summary.json`—the single source of truth—with full provenance (seed, exact command, platform, package versions, runtime, peak memory, and the `pauli_algebra_verified` flag). Nothing in the paper is typed by hand. The table is built by the pipeline `summary.json` → `make_tables.py` → `results/main_results.tex`, and the figures by `summary.json` → `make_figures.py` → the three `figures/*.pdf` files (`fig_schematic`, `fig_frontier`, `fig_family`). Figures are produced by `src/topoham/plotting.py` in *Nature Machine Intelligence* house style: vector PDF (`pdf.fonttype=42`, editable text), sans-serif, colour-blind-safe Okabe–Ito palette, no in-plot titles (the description lives in the caption), bold lower-case panel labels, and uncertainty on every quantitative panel (shaded 95% CI on the frontier, 95% CI error bars on the family bars). `fig_schematic` draws the Figure-1 method overview programmatically.

S10 How to reproduce everything

From `submission/code`, with the Conda environment active:

```
pip install -e .                          # or pip install -e ".[gpu]"
make test                                 # Pauli algebra (1e-9), Trotter
                                          # convergence, metrics, ordering claim
export KMP_DUPLICATE_LIB_OK=TRUE
bash scripts/reproduce_all.sh full        # writes summary.json, table, figures
```

The reported run (`configs/full.yaml`, seed 0): 120 instances (3 families $\times$ 5 sizes $n \in \{4, \ldots, 8\}$ $\times$ 8 seeded draws), time $t = 2.0$, fixed step budget $r = 3$, frontier grid $\{1, 2, 3, 4, 6, 8, 12\}$, mean gate proxy 48. Headline: commutator ordering raises mean fidelity from 0.205 ± 0.040 (random) to 0.548 ± 0.068, a $1.76\times$ infidelity reduction at the identical gate proxy, while cutting adjacent anticommuting pairs from 3.08 (random) to 0.28. The table and figure artifacts are copied verbatim into the `submission/` folder used to build this manuscript.

S11 What the result means (and what it does not)

Defensible: term ordering materially changes first-order Trotter fidelity at zero extra gate cost, and a commutator-graph ordering reaches the high-fidelity end of that range while minimising the adjacent anticommuting pairs that the leading error penalises (Sec. S5). The ordering-versus-random gap is large with disjoint confidence intervals—statistically clear. *Not claimed:* among the non-random orderings (commutator 0.548, coefficient 0.550, learned 0.549) the fidelity differences are *within* the confidence intervals; we do not claim the commutator policy is better than the coefficient heuristic on fidelity, and the learned router *tracks* rather than beats the best fixed strategy. On the dense molecular-like family the whole ordering effect is within noise at this budget—an honest negative result we report rather than omit. This is a clean, reproducible characterisation of a no-cost lever, not a claim of hardware advantage: first-order only, $n \leq 8$ (exact reference tractable), synthetic / structured Hamiltonians, classical CPU simulation, and a gate-count *proxy* $(L \cdot r)$.

References

[1] Richard P. Feynman. Simulating physics with computers. *International Journal of Theoretical Physics*, 21:467–488, 1982. doi: 10.1007/BF02650179.

[2] Seth Lloyd. Universal quantum simulators. *Science*, 273(5278):1073–1078, 1996. doi: 10.1126/science.273.5278.1073.

[3] Hale F. Trotter. On the product of semi-groups of operators. *Proceedings of the American Mathematical Society*, 10(4):545–551, 1959. doi: 10.1090/S0002-9939-1959-0108732-6.

[4] Masuo Suzuki. General theory of fractal path integrals with applications to many-body theories and statistical physics. *Journal of Mathematical Physics*, 32(2):400–407, 1991. doi: 10.1063/1.529425.

[5] Andrew M. Childs, Yuan Su, Minh C. Tran, Nathan Wiebe, and Shuchen Zhu. Theory of trotter error with commutator scaling. *Physical Review X*, 11(1):011020, 2021. doi: 10.1103/PhysRevX.11.011020.

[6] Andrew M. Childs, Yuan Su, Minh C. Tran, Nathan Wiebe, and Shuchen Zhu. Toward the first quantum simulation with quantum speedup. *Proceedings of the National Academy of Sciences*, 115(38):9456–9461, 2018. doi: 10.1073/pnas.1801723115.

[7] Naomichi Hatano and Masuo Suzuki. Finding exponential product formulas of higher orders. *Lecture Notes in Physics*, 679:37–68, 2005. doi: 10.1007/11526216_2.

[8] Andrew M. Childs, Yuan Su, Minh C. Tran, Nathan Wiebe, and Shuchen Zhu. A theory of trotter error. *Physical Review X*, 11(1):011020, 2021. doi: 10.1103/PhysRevX.11.011020.

[9] David Poulin, Matthew B. Hastings, Dave Wecker, Nathan Wiebe, Andrew C. Doberty, and Matthias Troyer. The trotter step size required for accurate quantum simulation of quantum chemistry. *Quantum Information & Computation*, 15(5–6):361–384, 2015.

[10] Vladyslav Verteletskyi, Tzu-Ching Yen, and Artur F. Izmaylov. Measurement optimization in the variational quantum eigensolver using a minimum clique cover. *The Journal of Chemical Physics*, 152(12):124114, 2020. doi: 10.1063/1.5141458.

[11] Pranav Gokhale, Olivia Angiuli, Yongshan Ding, Kaiwen Gui, Teague Tomesh, Martin Suchara, Margaret Martonosi, and Frederic T. Chong. $O(N^3)$ measurement cost for variational quantum eigensolver on molecular hamiltonians. *IEEE Transactions on Quantum Engineering*, 1:1–24, 2020. doi: 10.1109/TQE.2020.3035814.

[12] Ophelia Crawford, Billy van Straaten, Daochen Wang, Thomas Parks, Earl Campbell, and Stephen Brierley. Efficient quantum measurement of pauli operators in the presence of finite sampling error. *Quantum*, 5:385, 2021. doi: 10.22331/q-2021-01-20-385.

[13] Andrew Tranter, Peter J. Love, Florian Mintert, and Nathan Wiebe. Ordering of trotterization: Impact on errors in quantum simulation of electronic structure. *Entropy*, 21(12): 1218, 2019. doi: 10.3390/e21121218.

[14] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[15] Sergey B. Bravyi and Alexei Yu. Kitaev. Fermionic quantum computation. *Annals of Physics*, 298(1):210–226, 2002. doi: 10.1006/aphy.2002.6254.

[16] James D. Whitfield, Jacob Biamonte, and Alán Aspuru-Guzik. Simulation of electronic structure hamiltonians using quantum computers. *Molecular Physics*, 109(5):735–750, 2011. doi: 10.1080/00268976.2011.552441.

[17] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with numpy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[18] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. Scipy 1.0: fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[19] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008.

[20] Molena Huynh. Query-efficient quantum approximate optimization via graph-conditioned trust regions. *arXiv preprint arXiv:2604.24803*, 2026. doi: 10.48550/arXiv.2604.24803.

[21] Molena Huynh. Provable operator learning of size-transferable Trotter corrections for Hamiltonian simulation. Manuscript in preparation, 2026.

[22] Molena Huynh. Topology-conditioned warm starts match spectral initialization for depth-one QAOA. Manuscript in preparation, 2026.

[23] Molena Nguyen and Naihuan Jing. Zassenhaus expansion in solving the Schrödinger equation. *arXiv preprint arXiv:2505.09441*, 2025. doi: 10.48550/arXiv.2505.09441.

[24] Naihuan Jing and Molena Nguyen. Complexity bounds for Hamiltonian simulation in unitary representations. *arXiv preprint arXiv:2603.07231*, 2026. doi: 10.48550/arXiv.2603.07231.